

Travaux à présenter par les étudiants

Exposés thématiques & mini-projets intégrateurs

Quatre exposés (théorie + démonstration) et deux mini-projets pour mettre en pratique l'ensemble des piliers du cours.

Modalités

- Travail en **binôme ou trinôme** (3 étudiants maximum).
- Durée de passage : **15 minutes maximum**, démonstration comprise.
- Support attendu : un diaporama clair + **une démonstration** (en direct, ou enregistrée).
- Exposés (Partie A) : équilibrer la partie conceptuelle et la démo. Mini-projets (Partie B) : privilégier la démonstration de l'application déployée.
- Un sujet par groupe, à valider avec l'enseignant.
- La présentation des travaux aura lieu le 7 juillet.

Partie A — Exposés thématiques (théorie + démo)

Chaque exposé présente d'abord les concepts, puis illustre par une démonstration ciblée.

Sujet 1 — Backing Services : le Caching (Kubernetes & serverless)

Objectif : comprendre le rôle d'un cache distribué comme backing service, les stratégies de mise en cache, et comparer un cache déployé sur Kubernetes à un cache managé serverless.

À présenter

- Pourquoi mettre en cache : réduire la latence, soulager la base de données, absorber les pics de charge.
- Les caches distribués en mémoire : Redis et Memcached (modèle clé-valeur).
- Les stratégies : **cache-aside** (lazy loading), read-through, write-through / write-behind ; expiration (TTL), invalidation et politique d'éviction (LRU).
- Le cache comme backing service (Facteur IV) : accès via URL/identifiants, interchangeable.
- Cache sur Kubernetes (Redis via Helm / StatefulSet) vs cache managé serverless : **Amazon ElastiCache for Redis** (haute disponibilité multi-AZ, mise à l'échelle, maintenance gérée).
- Pièges : cohérence cache/base de données, « cache stampede », choix des données à mettre en cache.

Démonstration attendue

Créer un cluster **Amazon ElastiCache for Redis** (console ou CLI), s'y connecter avec `redis-cli`, illustrer SET/GET et un TTL, puis montrer le pattern **cache-aside** (lire depuis le cache, sinon depuis la source puis remplir le cache). Comparer rapidement avec un Redis déployé sur Kubernetes via Helm.

Outils suggérés : Amazon ElastiCache for Redis · `redis-cli` · Helm (bitnami/redis) · AWS CLI / console.

Sujet 2 — Backing Services : les Message Brokers (communication événementielle)

Objectif : comprendre la communication asynchrone et événementielle entre microservices, et le rôle des message brokers managés.

À présenter

- Communication **synchrone** (REST/gRPC) vs **asynchrone** (événementielle) : découplage, résilience et scalabilité.
- Files (queues) vs sujets (topics) : modèles point-à-point et **publish/subscribe**.
- Brokers : RabbitMQ, Apache Kafka ; services managés AWS : SQS (files), SNS (pub/sub), EventBridge.
- Patterns : publish/subscribe, fan-out, notification d'événement, idempotence et livraison « au moins une fois ».
- Focus : **Amazon SNS** comme topic pub/sub et ses abonnés (Lambda, SQS, e-mail).

Démonstration attendue

Créer un topic **Amazon SNS**, créer une fonction **AWS Lambda** et l'abonner au topic. Publier un message sur le topic et observer le déclenchement de la Lambda dans les **logs CloudWatch**. Illustrer le **fan-out** en ajoutant un second abonné (file SQS ou e-mail).

Outils suggérés : Amazon SNS · AWS Lambda · Amazon SQS · CloudWatch Logs · AWS CLI / console.

Sujet 3 — Pilier 7 : DevOps, CI/CD, IaC & GitOps (avec Terraform)

Objectif : présenter l'automatisation du cycle de vie logiciel (CI/CD), l'infrastructure as code et le GitOps, puis démontrer le workflow Terraform.

À présenter

- DevOps : culture et objectifs (réduire le délai entre le code et la production).
- CI/CD : intégration et déploiement continus ; étapes d'un pipeline (build, test, package, deploy) ; outils (GitHub Actions, GitLab CI, Jenkins).
- IaC : décrire l'infrastructure en code (déclaratif), idempotence, reproductibilité ; outils (Terraform, CloudFormation, Pulumi).
- GitOps : Git comme source unique de vérité, réconciliation continue (ArgoCD, Flux).
- Le workflow Terraform : `write` → `init` → `plan` → `apply` → `destroy` ; providers, state, variables, resources.

Démonstration attendue

Écrire un fichier Terraform provisionnant une instance **Amazon EC2** (provider "aws", resource "aws_instance"). Dérouler `terraform init`, `plan` puis `apply` ; vérifier l'instance dans la console AWS ; terminer par `terraform destroy` pour tout nettoyer.

Outils suggérés : Terraform · AWS CLI · compte AWS (EC2).

Sujet 4 — Pilier 8 : Monitoring & Observabilité (logging & tracing distribués)

Objectif : présenter l'observabilité d'un système distribué — métriques, logs et traces — et en faire la démonstration.

À présenter

- Monitoring vs observabilité ; les trois piliers : **métriques, logs, traces**.
- Métriques : **Prometheus** (collecte, PromQL) et **Grafana** (tableaux de bord, alertes).
- Logging distribué/centralisé : la pile **ELK** (Elasticsearch, Logstash, Kibana) ou Loki.
- Tracing distribué : **OpenTelemetry** et **Jaeger** ; notions de *trace* et de *span*, propagation du contexte entre microservices pour localiser la latence.
- Lien avec le service mesh (Istio) qui fournit une télémétrie automatique (Kiali, Jaeger, Grafana).

Démonstration attendue

Déployer **Prometheus + Grafana** (ou réutiliser la stack Istio / Kiali / Jaeger vue en cours) sur un cluster, générer du trafic, puis montrer un tableau de bord **Grafana** (métriques) et une **trace distribuée dans Jaeger** suivant une requête à travers plusieurs services. En variante : centralisation des logs dans Kibana.

Outils suggérés : Prometheus · Grafana · Jaeger · OpenTelemetry · (ELK / Loki) · Minikube / Istio.

Partie B — Mini-projets intégrateurs

Chaque mini-projet réutilise les notions vues dans l'ensemble des piliers. La présentation met l'accent sur la conception et la démonstration de l'application déployée.

Sujet 5 — Mini-projet : une application serverless sur AWS

Objectif : concevoir, déployer et démontrer une petite application entièrement serverless sur AWS, en réutilisant les concepts des piliers (backing services managés, communication événementielle, scalabilité, IaC, observabilité).

Ce que le projet doit intégrer

- Compute serverless : **AWS Lambda** exposé via **Amazon API Gateway**.
- Backing service managé : base serverless (**Amazon DynamoDB**) et/ou cache (ElastiCache).
- Communication événementielle : **SNS / SQS** ou EventBridge entre fonctions.
- Scalabilité automatique et scale-to-zero (propre au serverless).
- Déploiement par **IaC** (Terraform, AWS SAM ou Serverless Framework).
- Un minimum d'observabilité (journaux et métriques CloudWatch).

Cas d'usage possibles

Mini-API de commandes, de notifications ou de tâches : la création d'une ressource publie un événement qui déclenche une fonction de traitement (par exemple l'envoi d'une notification).

Démonstration attendue

Déployer l'application via IaC, appeler l'API (curl / Postman), montrer le flux événementiel et les journaux CloudWatch, puis illustrer la mise à l'échelle automatique.

Outils suggérés : AWS Lambda · API Gateway · DynamoDB · SNS / SQS / EventBridge · Terraform / AWS SAM / Serverless Framework · CloudWatch.

Sujet 6 — Mini-projet : une application microservices sur AWS

Objectif : concevoir, déployer et démontrer une application microservices conteneurisée sur AWS, en intégrant l'ensemble des piliers du cours.

Ce que le projet doit intégrer

- Plusieurs microservices conteneurisés (**Docker**), orchestrés par Kubernetes (**Amazon EKS avec Fargate**, ou Minikube).
- Communication **synchrone** (REST/gRPC, API Gateway / Ingress) et **asynchrone** (broker : Kafka ou SNS/SQS).
- Backing services : base de données (CockroachDB / RDS), cache (Redis / ElastiCache), broker de messages.
- Scalabilité : **HPA / KEDA**, ou mise à l'échelle serverless via Fargate.
- CI/CD et IaC : images poussées sur **ECR**, provisionnement via eksctl / Terraform.
- Observabilité : Prometheus / Grafana et tracing Jaeger (éventuellement via Istio).

Architecture suggérée

Deux à trois microservices (par exemple Product, Order, Notification) derrière une passerelle, communiquant en synchrone et en événementiel, et s'appuyant sur une base de données, un cache et un broker.

Démonstration attendue

Déployer la stack, montrer les microservices qui communiquent (synchrone + asynchrone), tester l'API, illustrer l'**autoscaling**, et présenter un tableau de bord d'observabilité (métriques + traces).

Outils suggérés : Docker · Amazon ECR · EKS / Fargate (ou Minikube) · Helm · Istio (option) · Kafka ou SNS/SQS · Redis / ElastiCache · CockroachDB / RDS · Prometheus / Grafana / Jaeger · Terraform / eksctl.